4-bit Unsigned Array Multiplier from RTL to GDSII

Digital VLSI Design Assignment Report

Udhav Malpani (2024112009) and Samyak Soni (2024112004)

Technology: SkyWater SKY130, library `sky130_fd_sc_hd` Top module: `mult_array`

Repository: https://github.com/rohday/rtl-to-gds

1 Part A: RTL Design and Verification

1.1 Design

The multiplier is described by two modules. The module `full_adder` implements a one-bit full adder with continuous assignments, `sum = a ^ b ^ cin` and `cout = (a & b) | (b & cin) | (a & cin)`. The module `mult_array` follows the connection table of Section 2 of the assignment. The operands are first captured in the 4-bit registers `a_reg` and `b_reg`. Sixteen two-input AND gates form the partial products $pp_i[j] = a_reg[j] \cdot b_reg[i]$. Row 0 contains only these gates, and rows 1 to 3 each contain a four-stage ripple-carry chain of `full_adder` instances, giving twelve instances in total. The eight-bit result is stored in the output register `p`. The $*$ operator is not used inside the design.

Both register groups use a rising-edge clock and an active-low asynchronous reset `rst_n`. The design therefore contains $4+4+8 = 16$ flip-flops. A pair of operands presented before clock edge 1 is captured into the input registers at that edge, and the corresponding product is captured into `p` at edge 2. The latency is two clock cycles and the throughput is one result per cycle. Figure 1 shows the structure.

1.2 Functional verification

The testbench `tb/tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first two cycles, and applies all $16 \times 16 = 256$ combinations of `a` and `b`. For each combination it waits for two rising edges and compares `p` with the expected value `a * b`. It prints each mismatch and a single summary line, and it writes `mult.vcd`. The RTL simulation produced

```
PASS: 256 vectors checked, 0 errors
```

To confirm that the testbench detects faults, the sum equation of `full_adder` was temporarily changed to `sum = a ^ b`. The summary line then became

```
FAIL: 256 vectors checked, 95 errors
```

The first reported mismatch was `a = 3, b = 3` (expected 9, obtained 5). This is the first operand pair in the enumeration order for which a nonzero carry-in reaches a full adder (here in row 1), so removing `cin` from the sum changes the result. The original equation was restored afterwards and the testbench again reported `PASS`.

1.3 Hand-worked example and waveform

The roll number of Udhav Malpani (2024112009) was used, so $A = 9 + 5 = 14 = 1110_2$ and $B = 7 = 0111_2$. The shift-and-add layout of the assignment gives

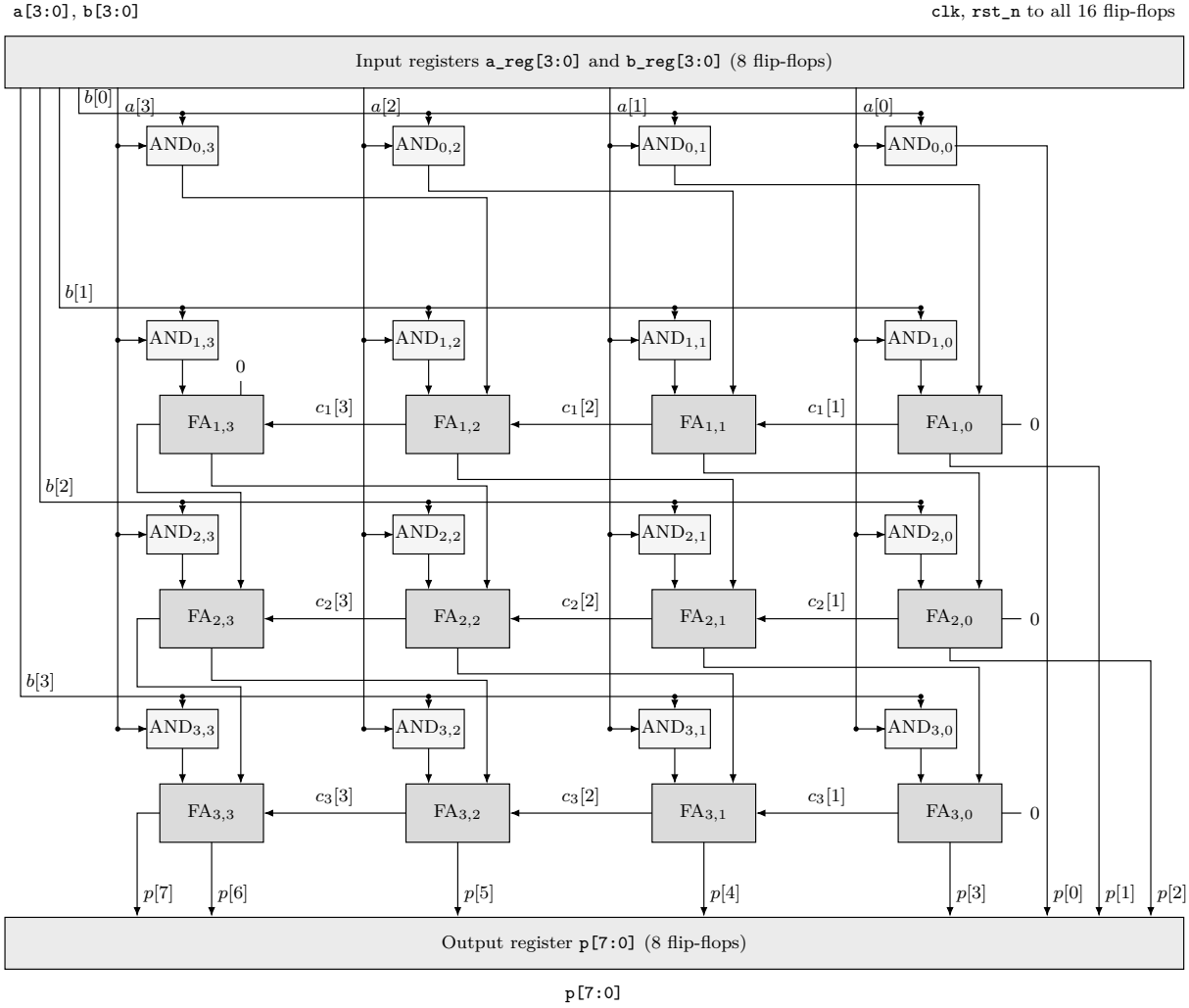


Figure 1: Gate-level block diagram of `mult_array`: 8 input flip-flops, 16 AND gates ($\text{AND}_{i,j} = a_reg[j] \cdot b_reg[i]$), 12 full adders ($\text{FA}_{i,j}$, rows 1 to 3) and 8 output flip-flops. Each full adder receives $pp_i[j]$ from the AND gate above it, the sum $r_{i-1}[j]$ from the previous row (diagonal connection) and the carry $c_i[j]$ from its right neighbour; the carry constants $c_i[0]$ are tied to 0.

```
      1 1 1 0     A   = 14
    x 0 1 1 1     B   =  7
  -------------
      1 1 1 0     pp0 (b0 = 1)
    1 1 1 0       pp1 (b1 = 1), shifted 1
  1 1 1 0         pp2 (b2 = 1), shifted 2
0 0 0 0           pp3 (b3 = 0), shifted 3
-----------------
0 1 1 0 0 0 1 0   P   = 98 = 0x62
```

Following the row structure of the array, row 0 gives $p[0] = 0$ and $r_0 = 0111$; row 1 computes $r_0 + pp_1 = 10101$, so $p[1] = 1$ and $r_1 = 1010$; row 2 computes $r_1 + pp_2 = 11000$, so $p[2] = 0$ and $r_2 = 1100$; and row 3 computes $r_2 + pp_3 = 01100$, so $p[3] = 0$ and $p[7:4] = 0110$. In the waveform of Figure 2 the operands $a = 14$, $b = 7$ are applied at 4637 ns, are captured into `a_reg` and `b_reg` at the clock edge at 4645 ns, and the product $p = 98$ (01100010_2) appears at the output register at the following edge at 4655 ns. This confirms the latency of two clock cycles.

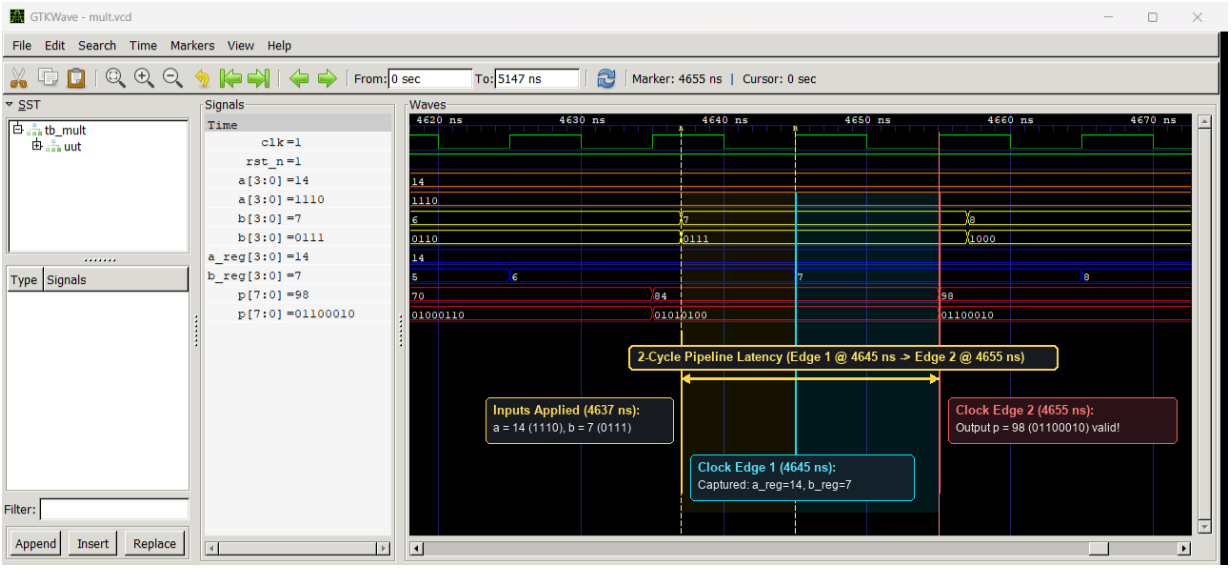


Figure 2: GTKWave view of `mult.vcd` for $A = 14$, $B = 7$, with the two-cycle latency marked.

2 Part B: Logic Synthesis with Yosys

The design was synthesised with `synth/synth.tcl` against the typical-corner library (file `sky130_fd_sc_hd__tt_025C_1v80.lib`, $T = 25\,^{\circ}\mathrm{C}$, 1.80 V). The full adders were flattened into the top module before technology mapping. The gate-level netlist was then simulated with the same testbench and the SKY130 cell models. Table 1 lists the results from `synth/mult_array_stat.rpt`.

Table 1: Part B results.

Metric	Value
Total number of cells	73 (16 flip-flops and 57 combinational cells)
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`, 25.024 $\mu\mathrm{m}^2$ each)
Total cell area	805.773 $\mu\mathrm{m}^2$
Flip-flop area	400.384 $\mu\mathrm{m}^2$ (16×25.024)
Flip-flop area (% of total)	49.69 %
Three most-used cell types	`dfrtp_1`: 16; `nand2_1`: 11; third place shared by `a21oi_1`, `and2_0`, `nand4_1` and `xnor2_1` with 4 cells each
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

3 Part C: Static Timing Analysis with OpenSTA

OpenSTA was run on the synthesised netlist at the typical corner with a 1.0 ns input delay on all inputs except `clk` and a 1.0 ns output delay on all outputs. With $T = 10.0$ ns the worst setup slack was +7.404 ns, so

$$T_{\min} = T - \mathrm{WNS} = 10.0 - 7.404 = 2.596 \text{ ns} \approx 2.60 \text{ ns}, \qquad F_{\max} = \frac{1000}{2.60} = 384.62 \text{ MHz}.$$

Repeating the analysis with $T = 2.60$ ns gave a setup slack of +0.004 ns (met), which is saved in `sta/mult_array_timing.rpt`. The results are collected in Table 2.

Table 2: Part C results.

Metric	Value
Worst slack at $T = 10$ ns	+7.404 ns
$T_{\min}$	2.60 ns (2.596 ns before rounding)
$F_{\max}$	384.62 MHz
Slack at $T = T_{\min}$	+0.004 ns (met)
Critical path start point	`_118_`, the flip-flop holding `b_reg[0]`
Critical path end point	`_112_`, the flip-flop driving `p[6]`
Cells on the critical path	8 combinational cells between the launch and capture flip-flops (10 including both flip-flops)
Latency, $2\,T_{\min}$	5.20 ns
Throughput	384.62 million results per second

The cells on the critical path, in order of traversal, are listed in Table 3. The sum of the listed delays is the arrival time of 2.460 ns, and the library setup time of −0.135 ns gives the required time of 2.465 ns.

Table 3: Critical path from `_118_` to `_112_` at $T = 2.60$ ns.

No.	Instance	Cell	Delay (ns)
0	`_118_` (launch)	`dfrtp_1` (CLK to Q)	0.395
1	`_083_`	`and2_0`	0.212
2	`_055_`	`or3_1`	0.361
3	`_059_`	`and4_1`	0.215
4	`_070_`	`o2111a_1`	0.234
5	`_076_`	`o21ai_0`	0.216
6	`_090_`	`xnor3_1`	0.274
7	`_091_`	`maj3_1`	0.375
8	`_092_`	`xor3_1`	0.178
9	`_112_` (capture)	`dfrtp_1` (setup −0.135)	–

Of the eight combinational cells, the first (`and2_0`) evaluates the partial product $a_reg[0]\cdot b_reg[0]$ and the remaining seven belong to the adder network; the last two (`maj3_1` and `xor3_1`) are the carry and sum functions of the final adder. After flattening, the mapper does not preserve the boundaries of the original full adders, so the correspondence between cells and adder stages is approximate. Nevertheless, eight combinational stages agree closely with the estimate of $2N = 8$ full-adder delays.

4 Part D: Physical Design with LibreLane

4.1 Clock period selection

From $T_{\min} = 2.60$ ns the starting period is $1.2 \times 2.60 = 3.12$ ns, rounded up to 3.5 ns. At this period the design had zero DRC and LVS errors and a typical-corner setup slack of +0.591 ns, but the slow corner `max_ss_100C_1v60` (100 °C, 1.60 V) had a setup slack of −2.144 ns. The period

was therefore increased in steps of 0.5 ns, requiring eight runs in total (3.5, 4.0, . . . , 7.0 ns). At 7.0 ns all nine signoff corners met setup and hold timing. The slack of every run is recorded in `README.md`. The final configuration is given in `pnr/config.json`, and the results are listed in Table 4.

Table 4: Part D results, taken from `pnr/metrics.json`.

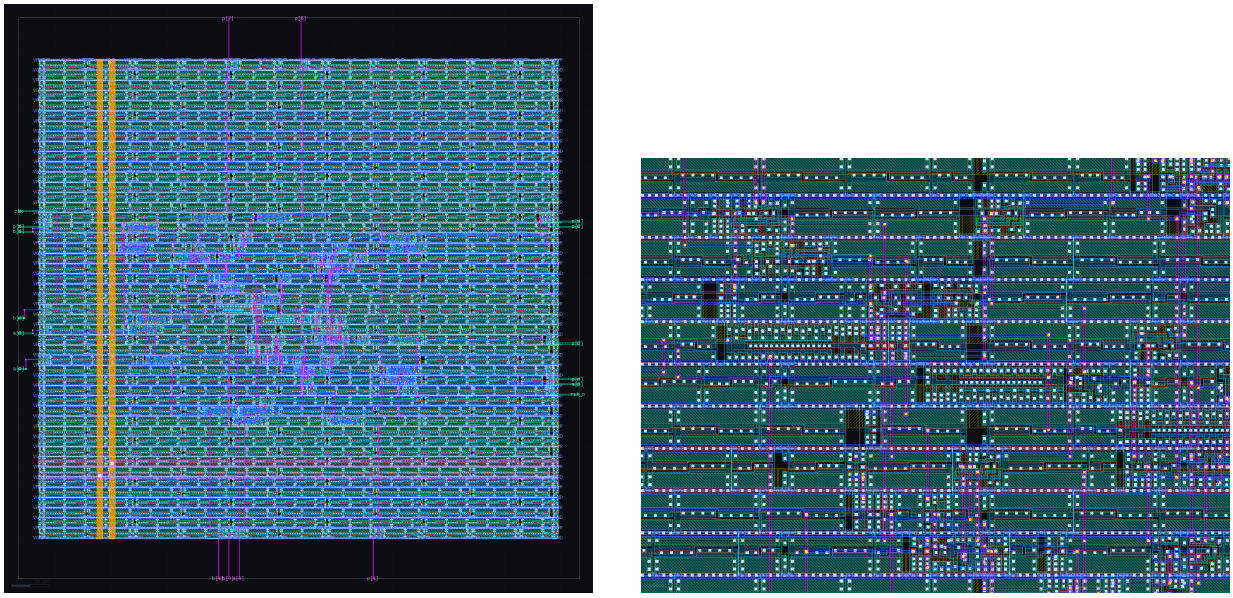
Metric	Value
`CLOCK_PERIOD` used	7.0 ns
Number of runs needed	8
Core area	17 759.5 μm^2 (die 150×150 μm $= 22\,500$ μm^2)
Cells after synthesis (Part B)	73
Cells after place and route	1 896 in total: 334 non-fill cells and 1 562 fill and decap cells
Kinds of cells added	245 tap cells; 19 timing-repair buffers (`buf_2`, `clkdlybuf4s50_1`); 3 clock buffers (`clkbuf_16`); 1 562 fill and decap cells (1 313 decap, 249 fill)
Wirelength	2 375 μm
Setup slack	+0.335 ns in the worst corner (`max_ss_100C_1v60`); +3.687 ns at the typical corner (`nom_tt_025C_1v80`)
Post-route $F_{\max}$	150.04 MHz (worst corner, $1000/(7.0 - 0.335)$); 301.82 MHz (typical corner, $1000/(7.0 - 3.687)$)
$F_{\max}$ from Part C	384.62 MHz
DRC errors	0 (Magic: 0, KLayout: 0)
LVS errors	0

5 Answers to the Questions of Section 10

Question 1. Synthesis produced $73 - 16 = 57$ cells for the logic. The count differs from $16 + 12 = 28$ for two reasons. A full adder is not a primitive cell of the library and is composed of several gates when mapped to two-input logic. In addition, the hierarchy was flattened before mapping, so the mapper optimised across the former module boundaries. Constants such as $c_1[0] = c_2[0] = c_3[0] = 0$ and $r_0[3] = 0$ were propagated, which reduces several of the first-stage adders to simpler functions, and shared sub-expressions were merged. The result is expressed in 57 library cells of different types, including compound cells such as `maj3_1`, `xor3_1`, `xnor3_1` and `o2111a_1`.

Question 2. The critical path starts at the input-register flip-flop `_118_` (`b_reg[0]`) and ends at the output-register flip-flop `_112_` (`p[6]`). Eight combinational cells lie between them: one AND gate and seven cells of the adder network (Table 3). For $N = 4$ the estimate of the longest ripple path is $2N = 8$ full-adder delays. The count of eight cells is consistent with this estimate, but the agreement is partly a consequence of the mapping, since compound cells such as `o2111a_1`, `maj3_1` and `xor3_1` implement more than one gate level each. The combinational delay is $2.460 - 0.395 = 2.065$ ns, i.e. approximately 0.26 ns per stage.

Question 3. Flip-flops occupy $400.384/805.773 = 49.69\,\%$ of the synthesised cell area. A `dfrtp_1` cell (25.024 μm^2) contains the master and slave latches, the clock inverters and the reset transistors, and is about $25.024/3.754 \approx 6.7$ times larger than a `nand2_1`. The number of registers



(a) Whole die (150 × 150 μm) with I/O pins and standard-cell rows.

(b) Zoomed region showing cell rows and the metal routing layers.

Figure 3: Final layout (`pnr/mult_array.gds`) displayed in KLayout.

grows as $4N = 16$, while the array contains only 57 mostly small gates. The ratio is therefore high at $N = 4$, because the combinational logic scales as N^2 and has not yet outgrown the fixed register overhead.

Question 4. Part C evaluates an ideal clock, zero wire parasitics and the typical corner. After routing, the netlist carries 2 375 μm of wire with resistance and capacitance, a propagated clock tree with a measured setup skew of about 0.265 ns, and additional buffers on the I/O nets. The post-route $F_{\max}$ at the typical corner is 301.82 MHz, compared with 384.62 MHz before layout, a reduction of about 22 %. The signoff result of 150.04 MHz is lower still because it is determined by the slowest corner (100 °C, 1.60 V), where the cell delays are considerably larger. The flow increased the cell count from 73 to 1 896 by adding: 3 clock buffers (`clkbuf_16`) so that the clock arrives at all 16 flip-flops with bounded skew and slew; 19 timing-repair buffers (`buf_2`, `clkdlybuf4s50_1`) to drive the I/O nets and the high-fanout `rst_n` net; 245 tap cells (`tapvpwrvgnd_1`), which connect the substrate and n-well to the supply rails at regular intervals to prevent latch-up; and 1 562 filler and decoupling cells, which complete the n-well and power-rail continuity of every row and provide local decoupling capacitance. Because the die is fixed at 150 × 150 μm the instance utilisation is only 6.90 %, and fill and decap cells therefore account for most of the added instances.

Question 5. A design rule check (DRC) verifies that the geometry of every mask layer in the GDSII file satisfies the foundry rules, for example minimum width, spacing and enclosure of metal and via shapes and antenna constraints. A layout-versus-schematic check (LVS) extracts the devices and connectivity from the layout and compares them with the post-route netlist, verifying that every device, pin and net corresponds to the intended circuit. Both checks are necessary before manufacturing: a DRC violation can make the geometry unmanufacturable or unreliable (shorts, opens or lithography failures), while an LVS mismatch means that a layout that can be fabricated implements a circuit different from the verified one. Our final layout has zero errors in both checks.

Question 6. An N-bit array multiplier requires N^2 AND gates and $N(N-1)$ full adders, and its critical path is approximately $2N$ full-adder delays. For $N = 4$ these values are 16, 12 and 8. For $N = 8$ the multiplier requires $8^2 = 64$ AND gates (four times as many) and $8 \cdot 7 = 56$ full adders ($56/12 \approx 4.7$ times as many, an increase of 44), and the critical path grows from 8 to 16 stages. Using the Part C numbers, the fixed overhead per path is the clock-to-output delay plus the setup time, $0.395 + 0.135 = 0.530$ ns, and the combinational delay per stage is $2.065/8 \approx 0.258$ ns. The estimated minimum period for $N = 8$ is

$$T_{\min}(8) \approx 0.530 + 16 \times 0.258 \approx 4.66 \text{ ns}, \qquad F_{\max}(8) \approx 215 \text{ MHz}.$$

The area therefore grows approximately quadratically with N, whereas the delay grows approximately linearly, so $T_{\min}$ increases by a factor of about 1.8 rather than 2 because the register overhead does not scale.